BHARATI VIDYAPEETH'S

INSTITUTE OF COMPUTER APPLICATIONS & MANAGEMENT

(Affiliated to Guru Gobind Singh Indraprastha University,

Approved by AICTE, New Delhi)

Database Management System (DBMS)

(MCA-165)

Practical File

Submitted To:

Dr. Rakhee

(Assistant Professor)

Submitted By:

Piyush Goel

(09311604425)

MCA 1st Sem, Sec

INDEX

<table>
<tr><th>S. No.</th><th>Problem Description</th><th>Date of Execution</th><th>Sign.</th></tr>
<tr><td>P1</td><td>Consider the following table: Write queries to perform the following:
Table : Book
<table>
<tr><th>Field Name</th><th>Data type</th><th>Size</th><th>Constraint</th></tr>
<tr><td>Book_Code</td><td>Varchar2</td><td>5</td><td>Primary key</td></tr>
<tr><td>ISBN No.</td><td>Varchar2</td><td>8</td><td>Not Null/</td></tr>
<tr><td>Book_Name</td><td>Varchar2</td><td>8</td><td>Unique</td></tr>
<tr><td>Publisher</td><td>Varchar2</td><td>6</td><td></td></tr>
<tr><td>Price</td><td>Number</td><td>5,2</td><td>>=100</td></tr>
<tr><td>Author_Name</td><td>Varchar2</td><td>8</td><td></td></tr>
<tr><td>Date_of_Launch</td><td>Date</td><td></td><td></td></tr>
</table>
1. Insert few records in it.
2. Increase the size of the Book_Name field to 10 characters.
3. Display the total price of all the books of “TMH’ publishing house.
4. Display the details of all the books of author “Yashwant Kanetkar” and “Robert Lafore”.
5. Delete all the books belonging to “PHI” publisher.
6. Update the price of books by 5% which belong to ‘BPB’ publishers.
7. Create a new table “Author” with author details such as author id (primary key), Name of the author, subject, contact details, research area etc.
8. Insert records in it and display the records of those authors who have ‘Web mining’ as their research area.
9. Add a new column “Author_Id” in “Book” table and make it a foreign key on “author id” of “Author” table.
10. Update all the previous records of Book table to add information of author id (newly added column)
11. Display the records of the books where book name starts with ‘C’ or ‘G’ or ‘K’.
12. Create a view for the user to access the Book Code, Name and Price of books which are published by ‘Pearson Education’</td><td></td><td></td></tr>
<tr><td>P2</td><td>Consider the following table:
1. Insert few records except total marks.
2. Create a view on the table to access roll no, name and total marks of those students who belong to ‘Delhi’.
3. Calculate total marks of each student and save it.
4. Display the name of the student who has got the highest</td><td></td><td></td></tr>
</table>

marks.

5. Display the records of those students who have got marks more than the marks of ‘Kiran’.

6. Display the percentage of all the students with their names. Laboratory Manual for MCA-165; Database Management Systems Lab. © Bharati Vidyapeeth’s Institute of Computer Applications and Management (BVICAM), New Delhi | Page 14 of 18

7. Display the names of those students who have got maximum marks in sub1 and more than 50 in sub2 and sub3.

8. Update the marks of all those students who are fail, with grace marks 5. (fail means less than 40)

9. Delete the record of student who got minimum marks.

10. Display record of student who got second highest marks.

11. Display the average total marks of all the student's city wise excluding the students of ‘Gurgaon’.

12. Get the names of students with Roll_no less than 30 and whose total marks is more than the total marks of at least one student with Roll_no greater than or equal to 30.

Table : Student

Field Name	Data type	Size	Constraint
Roll_no	Number	2	Primary Key
Name	Varchar2	20	Not Null
Address	Varchar2	50	
City	Varchar2	30	
Marks_Sub1	Number	5,2	>=0 and <=100
Marks_sub2	Number	5,2	>=0 and <=100
Marks_sub3	Number	5,2	>=0 and <=100
Total	Number	5,2	>=0 and <=300

P3

Consider the following three relations: Primary Keys are indicated by # Foreign Keys are underlined. Employee (Eid#, EName, Address, DateOfJoining, Department) Project (Pid#, PName, StartDate, TerminationDate) AssignedTo (Eid, Pid) Now write queries for the following in relational algebra as well as in SQL:

1. Find the employees working on ‘Banking’ project.
2. Find the projects assigned to the employees of D01 or D02.
3. Find the employees who belong to Delhi and work on either ‘University’project or ‘ShareMarket’ project
4. Find the employees who do not work on ‘University’ project.
5. Find the employees who work on all projects.
6. List the employees who have not been assigned any project.
7. Find the employees who joined the department after the commencement of ‘Bank’ project.
8. Display the start and termination date of projects which are

	allotted to 'Jai Prakash'		
P4	Consider the following relations: Primary Keys are indicated by # Foriegn Keys are underlined. Student (Roll#, SName, City, Age, Gender) Course (Cid#, CName, Semester, Credits, Fee) Enrollment (Roll, Cid, DateOfReg) Now write SQL queries for the following: 1. Find the details of students registered in MCA course. 2. Display the total number of students enrolled in MBA. 3. Display the names of students who are enrolled in the course n which "Anita" is registered. 4. Display the names of all the students with their date of enrollment in the courses. 5. Display the name of the course in which "Kunal" is registered. 6. Display the names of boys-students enrolled in MCA. 7. Display the number of students registered in MCA course who live outside Delhi.		
P5	Consider any table of your choice and execute following functions on numeric, character and date type fields : • sqrt( ) • power( ) • ceil( ) • max( ) • min( ) • avg( ) • count( ) • exp( ) • mod( ) • sum ( ) • floor( ) • abs( ) • greatest ( ) • round( ) • to_date( ) • months_between( ) • last_day( ) • add_months( ) • next_day( ) • sysdate • round( ) • to_char( ) length( ) • substr( ) • lower( ) • initcap( ) • instr( ) • concat( ) • lpad( ) • rpad( ) • ltrim( ) • rtrim( ) • replace( )		
P6	Write a program that demonstrates dynamic binding by creating an array of `Person` references, storing both `Patient` and `Doctor` objects, and calling `displayDetails()`..		
P7	Write a PL/SQL code that will reverse a given number like 12345 to 54321.		
P8	Write a PL/SQL code to accept emp_id from user and display his / her record. Table: Employee (EmpId, EName, DOJ, DOB, Salary, Address)		
P9	The bank manager of Delhi branch decides to mark the status of all those accounts as 'Inactive' on which there are no transactions performed in last 6 months. Whenever any such update takes place the corresponding record is inserted in another table 'InactiveAccounts' with the name and account no of the account holder and his balance. Table: Account (AccNo, Name, Address, PANNo, Mobile, Status)		

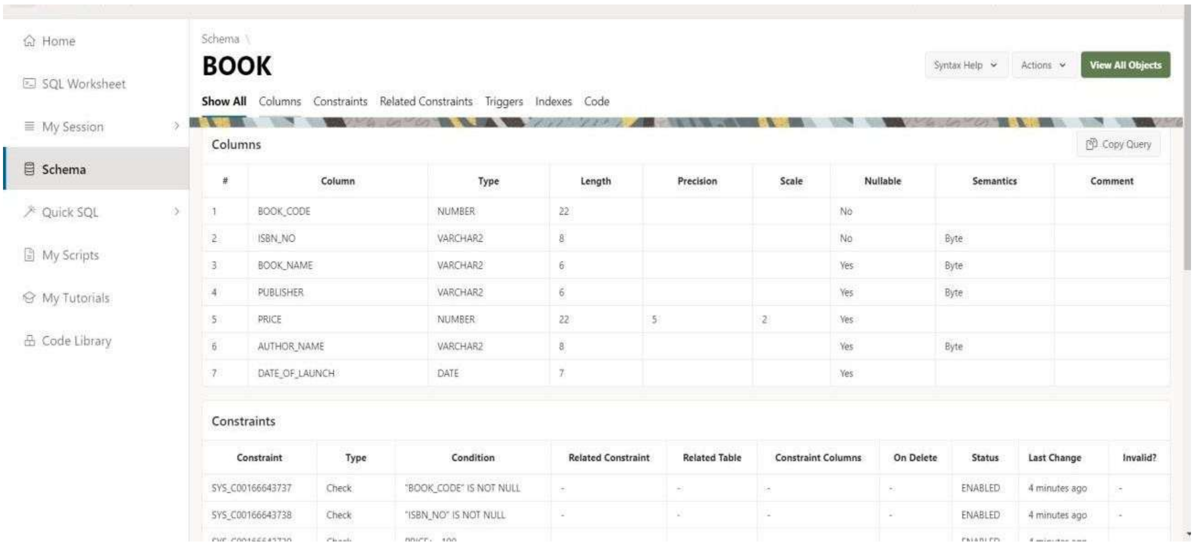
P10	Write a Cursor(PL/SQL code) to display the Employee_name, DateofBirth, and Designation whose basic salary is greater than 15000, if not found then show the proper error message. (Use Exception handing) Table : Employee(EmpId, EName, DOJ, DOB, Salary, Address)		
P11	Create a function that accepts emp id from the calling procedure and returns his salary. Table: Employee (EmpId, EName, DOJ, DOB, Salary, Address) Create a function that accepts emp id from the calling procedure and returns his salary. Table: Employee (EmpId, EName, DOJ, DOB, Salary, Address)		
P12	Write a PL/SQL function ODDEVEN to return value TRUE if the number passed to it is EVEN else will return FALSE		

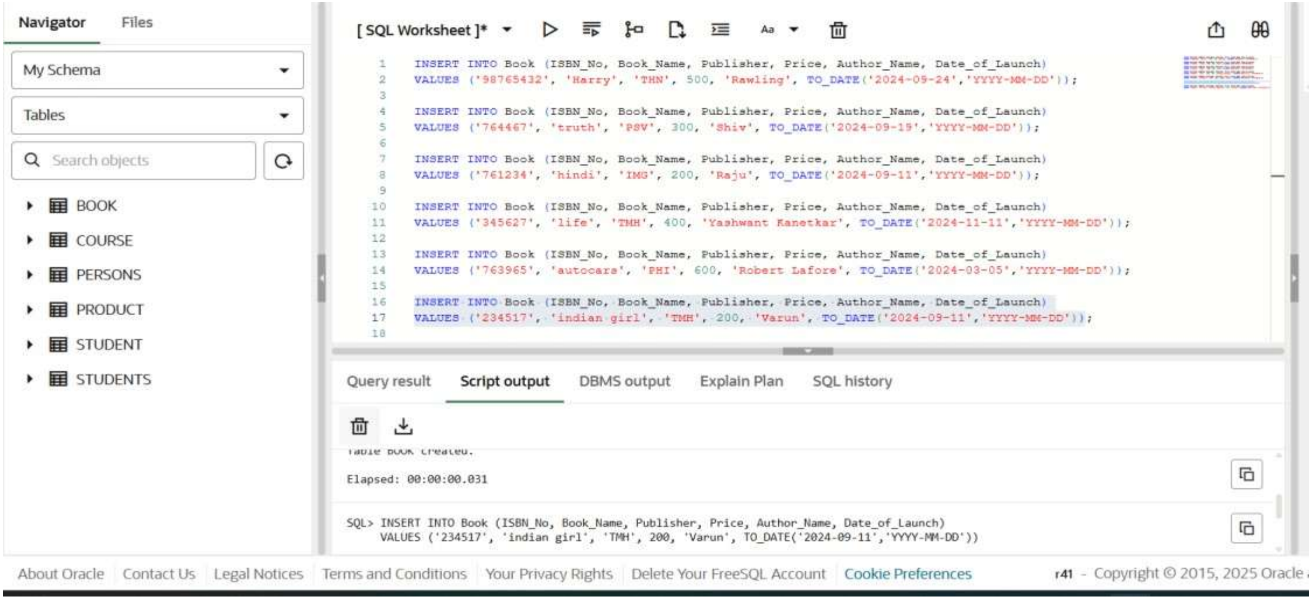
P1 Consider the following table:

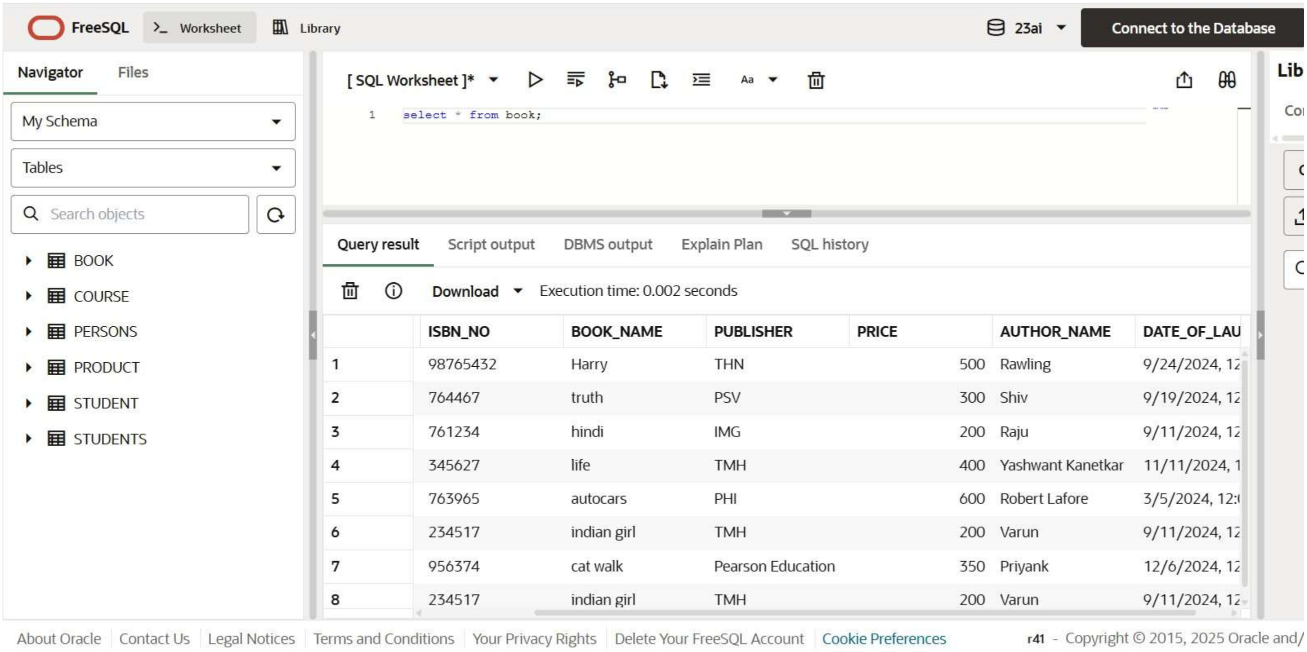
Table : Book

Field Name	Data type	Size	Constraint
Book_Code	Varchar2	5	Primary key
ISBN No.	Varchar2	8	Not Null/
Book_Name	Varchar2	8	Unique
Publisher	Varchar2	6	
Price	Number	5,2	>=100
Author_Name	Varchar2	8	
Date_of_Launch	Date		

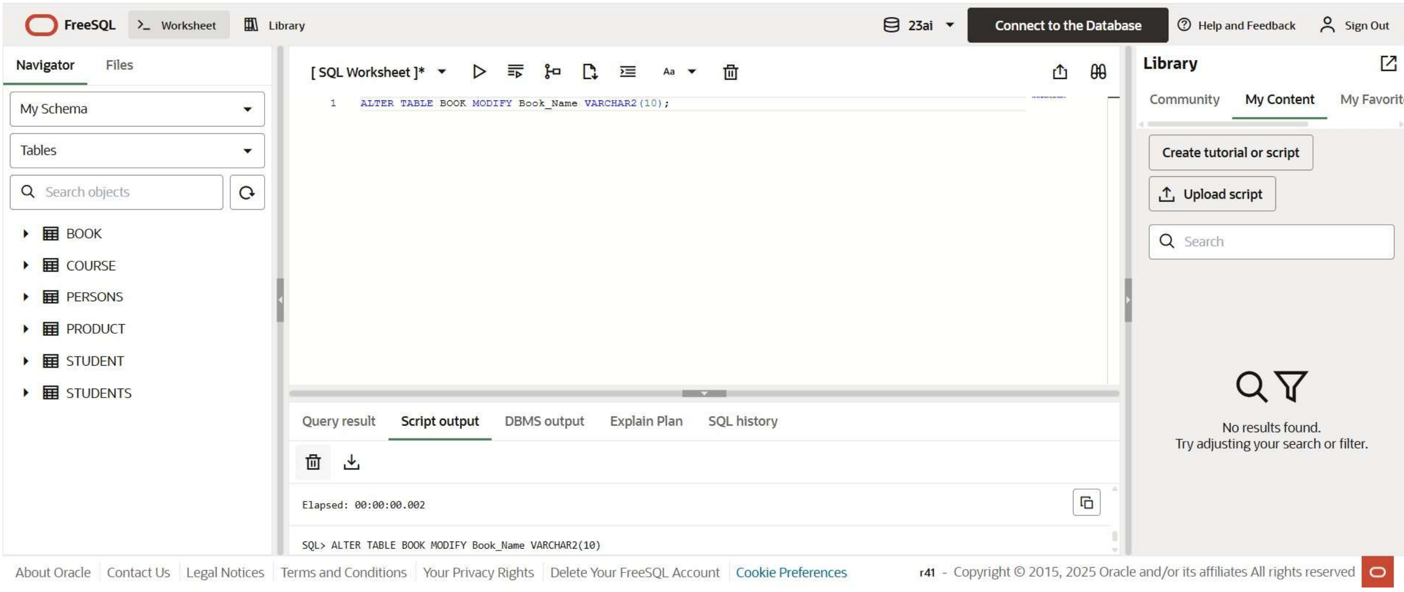
Write queries to perform the following:

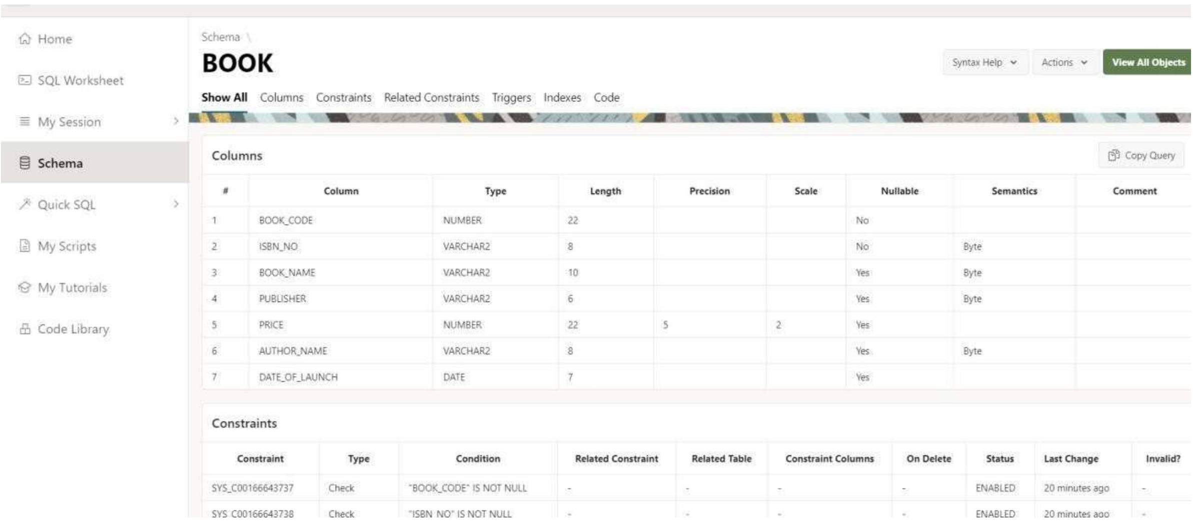



Navigator
Files
My Schema
Tables
Search objects
BOOK
COURSE
PERSONS
PRODUCT
STUDENT
STUDENTS
[SQL Worksheet]*
INSERT INTO Book (ISBN_No, Book_Name, Publisher, Price, Author_Name, Date_of_Launch)
VALUES ('98765432', 'Harry', 'THN', 500, 'Rawling', TO_DATE('2024-09-24','YYYY-MM-DD'));
INSERT INTO Book (ISBN_No, Book_Name, Publisher, Price, Author_Name, Date_of_Launch)
VALUES ('764467', 'truth', 'PSV', 300, 'Shiv', TO_DATE('2024-09-19','YYYY-MM-DD'));
INSERT INTO Book (ISBN_No, Book_Name, Publisher, Price, Author_Name, Date_of_Launch)
VALUES ('761234', 'hindi', 'IMG', 200, 'Raju', TO_DATE('2024-09-11','YYYY-MM-DD'));
INSERT INTO Book (ISBN_No, Book_Name, Publisher, Price, Author_Name, Date_of_Launch)
VALUES ('345627', 'life', 'TMH', 400, 'Yashwant Kanetkar', TO_DATE('2024-11-11','YYYY-MM-DD'));
INSERT INTO Book (ISBN_No, Book_Name, Publisher, Price, Author_Name, Date_of_Launch)
VALUES ('763965', 'autocars', 'PHI', 600, 'Robert Lafore', TO_DATE('2024-03-05','YYYY-MM-DD'));
INSERT INTO Book (ISBN_No, Book_Name, Publisher, Price, Author_Name, Date_of_Launch)
VALUES ('234517', 'indian girl', 'TMH', 200, 'Varun', TO_DATE('2024-09-11','YYYY-MM-DD'));
Query result
Script output
DBMS output
Explain Plan
SQL history
Elapsed: 00:00:00.031
SQL> INSERT INTO Book (ISBN_No, Book_Name, Publisher, Price, Author_Name, Date_of_Launch)
VALUES ('234517', 'indian girl', 'TMH', 200, 'Varun', TO_DATE('2024-09-11','YYYY-MM-DD'))
About Oracle
Contact Us
Legal Notices
Terms and Conditions
Your Privacy Rights
Delete Your FreeSQL Account
Cookie Preferences
r41 - Copyright © 2015, 2025 Oracle

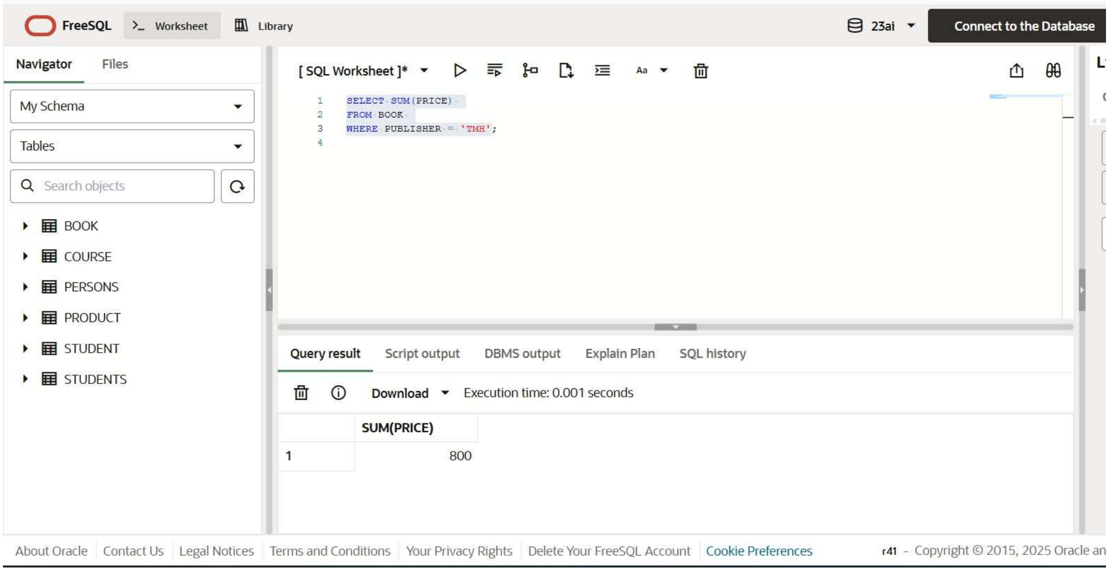

FreeSQL
Worksheet
Library
23ai
Connect to the Database
Navigator
Files
My Schema
Tables
Search objects
BOOK
COURSE
PERSONS
PRODUCT
STUDENT
STUDENTS
[SQL Worksheet]*
select * from book;
Query result
Script output
DBMS output
Explain Plan
SQL history
Download
Execution time: 0.002 seconds
ISBN_NO BOOK_NAME PUBLISHER PRICE AUTHOR_NAME DATE_OF_LAU
1 98765432 Harry THN 500 Rawling 9/24/2024, 12
2 764467 truth PSV 300 Shiv 9/19/2024, 12
3 761234 hindi IMG 200 Raju 9/11/2024, 12
4 345627 life TMH 400 Yashwant Kanetkar 11/11/2024, 1
5 763965 autocars PHI 600 Robert Lafore 3/5/2024, 12:
6 234517 indian girl TMH 200 Varun 9/11/2024, 12
7 956374 cat walk Pearson Education 350 Priyank 12/6/2024, 12
8 234517 indian girl TMH 200 Varun 9/11/2024, 12
About Oracle
Contact Us
Legal Notices
Terms and Conditions
Your Privacy Rights
Delete Your FreeSQL Account
Cookie Preferences
r41 - Copyright © 2015, 2025 Oracle and/

Q2. Increase the size of the Book_Name field to 10 characters.

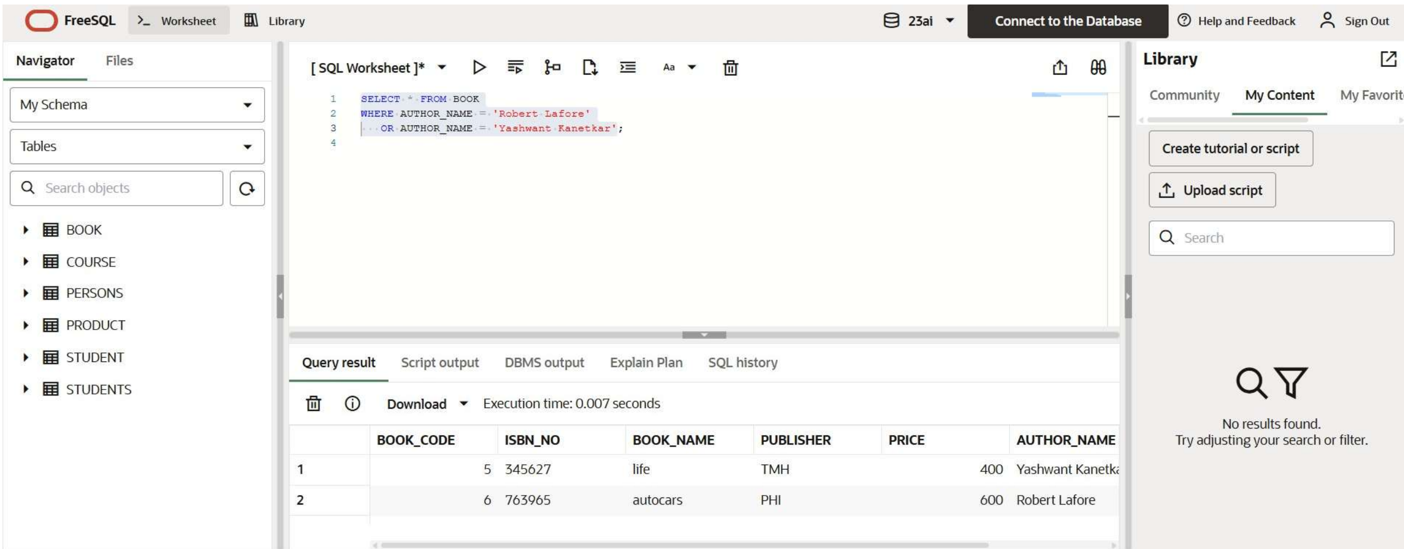




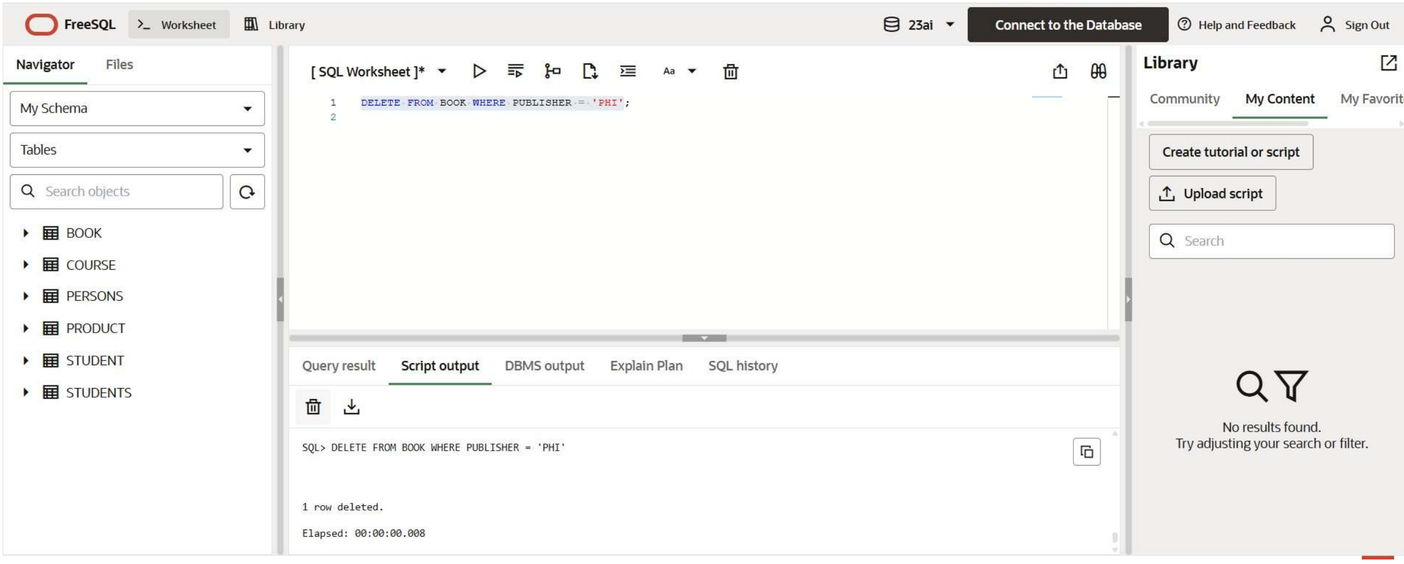
Q3.Display the total price of all the books of “TMH’ publishing house



Q4. Display the details of all the books of author “Yashwant Kanetkar” and “Robert Lafore”

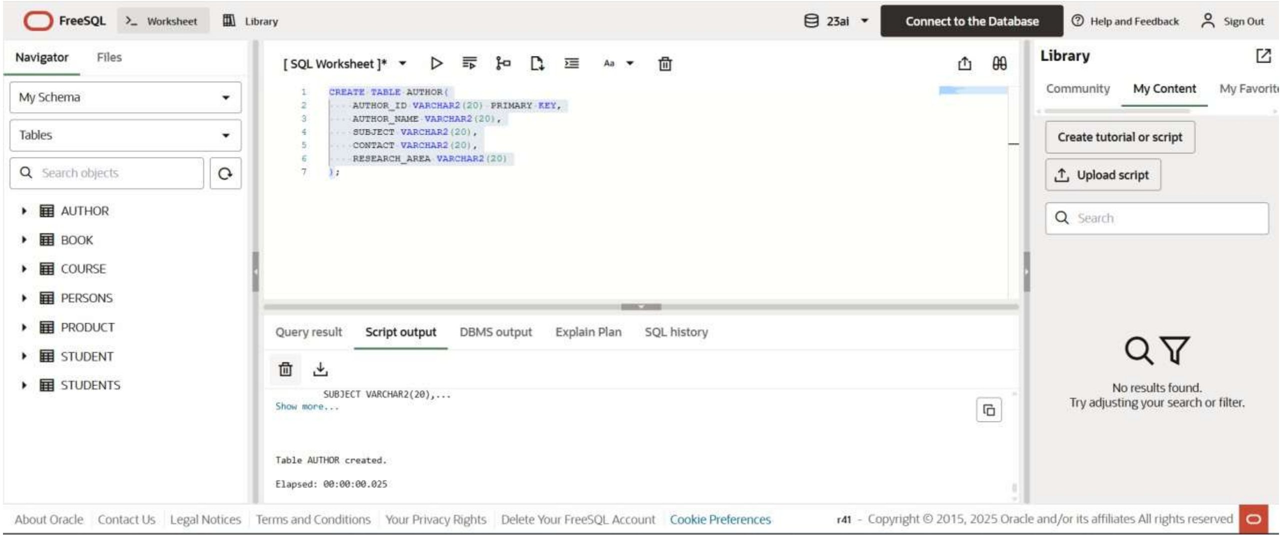


Q5. Delete all the books belonging to “PHI” publisher.

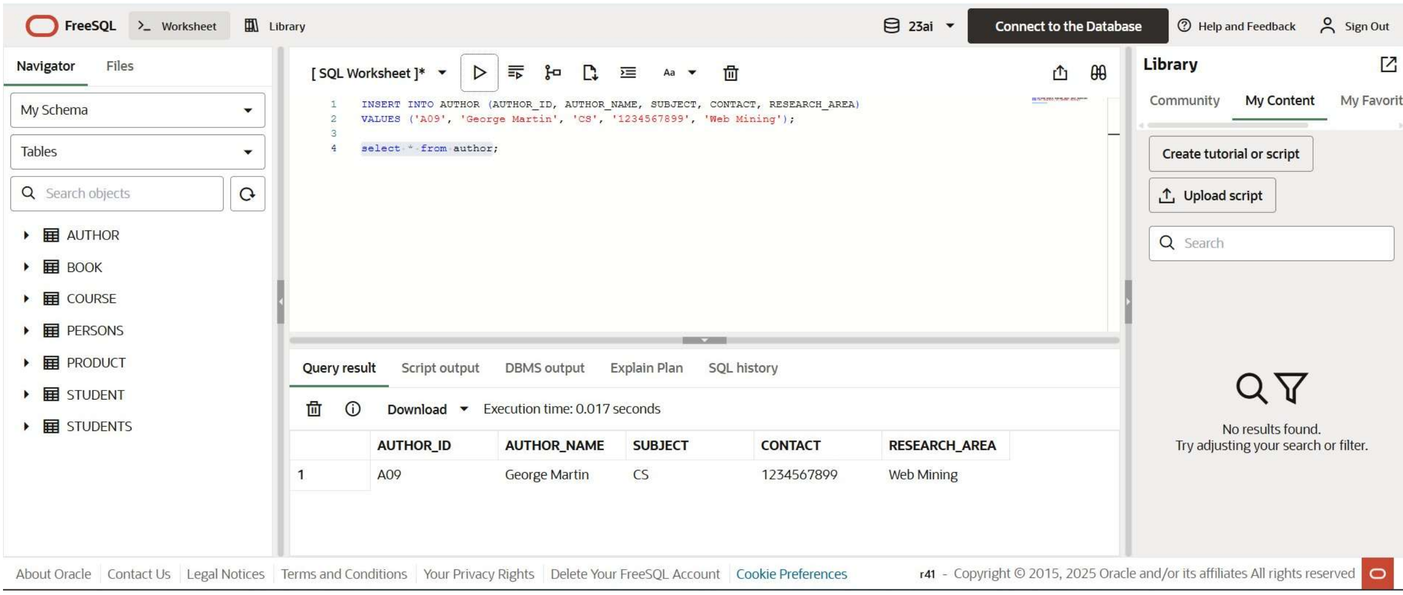


Q6. Update the price of books by 5% which belong to ‘BPB’ publishers.

Q-7 Create a new table “Author” with author details such as author id (primary key), Name of the author, subject, contact details, research area etc.



Q-8 Insert records in it and display the records of those authors who have ‘Web mining’ as their research area.



Q-9 Add a new column “Author_Id” in “Book” table and make it a foreign key on “author id” of “Author” table.

Q-10 Update all the previous records of Book table to add information of author id (newly added column)

Q-11 Display the records of the books where book name starts with ‘C’ or ‘G’ or ‘K’.

Q-12 Create a view for the user to access the Book Code, Name and Price of books which are published by ‘Pearson Education’.

P2 Consider the following table:

Table : Student

Field Name	Data type	Size	Constraint
Roll_no	Number	2	Primary Key
Name	Varchar2	20	Not Null
Address	Varchar2	50	
City	Varchar2	30	
Marks_Sub1	Number	5,2	>=0 and <=100
Marks_sub2	Number	5,2	>=0 and <=100
Marks_sub3	Number	5,2	>=0 and <=100
Total	Number	5,2	>=0 and <=300

```
CREATE TABLE Student (
    Roll_no INT PRIMARY KEY,
    Name VARCHAR(20) NOT NULL,
    Address VARCHAR(50),
    City VARCHAR(30),
    Marks_sub1 DECIMAL(5,2) CHECK (Marks_sub1 >= 0 AND Marks_sub1 <= 100),
    Marks_sub2 DECIMAL(5,2) CHECK (Marks_sub2 >= 0 AND Marks_sub2 <= 100),
    Marks_sub3 DECIMAL(5,2) CHECK (Marks_sub3 >= 0 AND Marks_sub3 <= 100),
    Total DECIMAL(5,2) CHECK (Total >= 0 AND Total <= 300)
);

SELECT *
FROM Student;
```

Roll_no	Name	Address	City	Marks_sub1	Marks_sub2	Marks_sub3	Total
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

1. Insert few records except total marks.

```
INSERT INTO Student (Roll_no, Name, Address, City, Marks_sub1, Marks_sub2, Marks_sub3)
VALUES
(11, 'Kiran', 'A-12', 'Delhi',    78, 67, 89),
(12, 'Aman',  'B-11', 'Delhi',    55, 49, 65),
(13, 'Rohit', 'C-22', 'Mumbai',   80, 81, 79),
(14, 'Neha',  'D-09', 'Gurgaon',  39, 45, 50),
(15, 'Simran','E-77', 'Delhi',    90, 88, 92),
(16, 'Vikas', 'F-67', 'Kolkata',  45, 60, 58);

SELECT *
FROM Student;
```

Roll_no	Name	Address	City	Marks_sub1	Marks_sub2	Marks_sub3	Total
11	Kiran	A-12	Delhi	78.00	67.00	89.00	NULL
12	Aman	B-11	Delhi	55.00	49.00	65.00	NULL
13	Rohit	C-22	Mumbai	80.00	81.00	79.00	NULL
14	Neha	D-09	Gurgaon	39.00	45.00	50.00	NULL
15	Simran	E-77	Delhi	90.00	88.00	92.00	NULL
16	Vikas	F-67	Kolkata	45.00	60.00	58.00	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

2. Create a view on the table to access roll no, name and total marks of those students who belong to 'Delhi'.

```
CREATE VIEW Delhi_Students AS
SELECT Roll_no, Name, Total
FROM Student
WHERE City = 'Delhi';
```

3. Calculate total marks of each student and save it.

```
UPDATE Student
SET Total = Marks_sub1 + Marks_sub2 + Marks_sub3;

SELECT *
FROM Student;
```

Roll_no	Name	Address	City	Marks_sub1	Marks_sub2	Marks_sub3	Total
11	Kiran	A-12	Delhi	78.00	67.00	89.00	234.00
12	Aman	B-11	Delhi	55.00	49.00	65.00	169.00
13	Rohit	C-22	Mumbai	80.00	81.00	79.00	240.00
14	Neha	D-09	Gurgaon	39.00	45.00	50.00	134.00
15	Simran	E-77	Delhi	90.00	88.00	92.00	270.00
16	Vikas	F-67	Kolkata	45.00	60.00	58.00	163.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

4. Display the name of the student who has got the highest marks.

```sql
SELECT Name
FROM Student
ORDER BY Total DESC
LIMIT 1;
```

Name
Simran

5. Display the records of those students who have got marks more than the marks of ‘Kiran’.

```sql
SELECT *
FROM Student
WHERE Total > (SELECT Total FROM Student WHERE Name = 'Kiran');
```

Roll_no	Name	Address	City	Marks_sub1	Marks_sub2	Marks_sub3	Total
13	Rohit	C-22	Mumbai	80.00	81.00	79.00	240.00
15	Simran	E-77	Delhi	90.00	88.00	92.00	270.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

6. Display the percentage of all the students with their names.

```sql
SELECT Name, (Total / 300) * 100 AS Percentage
FROM Student;
```

	Name	Percentage
	Kiran	78.000000
	Aman	56.333300
	Rohit	80.000000
	Neha	44.666700
	Simran	90.000000
	Vikas	54.333300

7. Display the names of those students who have got maximum marks in sub1 and more than 50 in sub2 and sub3.

```
SELECT Name
FROM Student
WHERE Marks_sub1 = (SELECT MAX(Marks_sub1) FROM Student)
  AND Marks_sub2 > 50
  AND Marks_sub3 > 50;
```

	Name
	Simran

8. Update the marks of all those students who are fail, with grace marks 5. (fail means less than 40)

```sql
UPDATE Student
SET Marks_sub1 = Marks_sub1 + IF(Marks_sub1 < 40, 5, 0),
    Marks_sub2 = Marks_sub2 + IF(Marks_sub2 < 40, 5, 0),
    Marks_sub3 = Marks_sub3 + IF(Marks_sub3 < 40, 5, 0);

UPDATE Student
SET Total = Marks_sub1 + Marks_sub2 + Marks_sub3;
```

9. Delete the record of student who got minimum marks.

```sql
DELETE FROM Student
ORDER BY Total ASC
LIMIT 1;


SELECT *
FROM Student;
```

Roll_no	Name	Address	City	Marks_sub1	Marks_sub2	Marks_sub3	Total
11	Kiran	A-12	Delhi	78.00	67.00	89.00	234.00
12	Aman	B-11	Delhi	55.00	49.00	65.00	169.00
13	Rohit	C-22	Mumbai	80.00	81.00	79.00	240.00
15	Simran	E-77	Delhi	90.00	88.00	92.00	270.00
16	Vikas	F-67	Kolkata	45.00	60.00	58.00	163.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

10. Display record of student who got second highest marks.

```sql
SELECT *
FROM Student
ORDER BY Total DESC
LIMIT 1 OFFSET 1;
```

Roll_no	Name	Address	City	Marks_sub1	Marks_sub2	Marks_sub3	Total
13	Rohit	C-22	Mumbai	80.00	81.00	79.00	240.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

11. Display the average total marks of all the student's city wise excluding the students of ‘Gurgaon’.

```
SELECT City, AVG(Total) AS Avg_Total
FROM Student
WHERE City <> 'Gurgaon'
GROUP BY City;
```

City	Avg_Total
Delhi	224.333333
Mumbai	240.000000
Kolkata	163.000000

12. Get the names of students with Roll_no less than 30 and whose total marks is more than the total marks of at least one student with Roll_no greater than or equal to 30.

```
SELECT *
FROM Student s
WHERE s.Roll_no < 30
  AND s.Total > (
        SELECT MIN(Total)
        FROM Student
        WHERE Roll_no >= 30
    );
```

Roll_no	Name	Address	City	Marks_sub1	Marks_sub2	Marks_sub3	Total
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

P3 Consider the following three relations:
Primary Keys are indicated by #
Foreign Keys are underlined.
Employee (Eid#, EName, Address, DateOfJoining, Department)
Project (Pid#, PName, StartDate, TerminationDate)
AssignedTo (Eid, Pid)

```
CREATE TABLE Employee (
    Eid INT PRIMARY KEY,
    EName VARCHAR(50),
    Address VARCHAR(100),
    DateOfJoining DATE,
    Department VARCHAR(10)
);
INSERT INTO Employee VALUES
(1, 'Amit', 'Delhi', '2020-02-10', 'D01'),
(2, 'Rohit', 'Mumbai', '2019-05-12', 'D02'),
(3, 'Jai Prakash', 'Delhi', '2021-04-01', 'D03'),
(4, 'Anita', 'Delhi', '2018-12-20', 'D01');

SELECT *
FROM Employee;
```

Eid	EName	Address	DateOfJoining	Department
1	Amit	Delhi	2020-02-10	D01
2	Rohit	Mumbai	2019-05-12	D02
3	Jai Prakash	Delhi	2021-04-01	D03
4	Anita	Delhi	2018-12-20	D01
NULL	NULL	NULL	NULL	NULL

```sql
CREATE TABLE Project (
    Pid INT PRIMARY KEY,
    PName VARCHAR(50),
    StartDate DATE,
    TerminationDate DATE
);
INSERT INTO Project VALUES
(101, 'Banking', '2020-01-01', '2023-12-31'),
(102, 'University', '2019-03-10', '2022-05-01'),
(103, 'ShareMarket', '2018-11-05', '2023-10-10'),
(104, 'Bank', '2020-02-01', '2024-10-01');

SELECT *
FROM Project;
```

Pid	PName	StartDate	TerminationDate
101	Banking	2020-01-01	2023-12-31
102	University	2019-03-10	2022-05-01
103	ShareMarket	2018-11-05	2023-10-10
104	Bank	2020-02-01	2024-10-01
NULL	NULL	NULL	NULL

```sql
CREATE TABLE AssignedTo (
    Eid INT,
    Pid INT,
    FOREIGN KEY (Eid) REFERENCES Employee(Eid),
    FOREIGN KEY (Pid) REFERENCES Project(Pid)
);
INSERT INTO AssignedTo VALUES
(1, 101),
(1, 102),
(2, 101),
(3, 102),
(3, 103),
(4, 103);

SELECT *
FROM AssignedTo;
```

Eid	Pid
1	101
1	102
2	101
3	102
3	103
4	103

Now write queries for the following in relational algebra as well as in SQL:
1. Find the employees working on 'Banking' project.

BANK ← σ(PName = 'Banking')(Project)
EMP_PROJ ← BANK ⋈ AssignedTo ⋈ Employee
π(Eid, EName)(EMP_PROJ)

```sql
SELECT E.Eid, E.EName
FROM Employee E
JOIN AssignedTo A ON E.Eid = A.Eid
JOIN Project P ON A.Pid = P.Pid
WHERE P.PName = 'Banking';
```

	Eid	EName
	1	Amit
	2	Rohit

2. Find the projects assigned to the employees of D01 or D02.

EMP ← σ(Department = 'D01' OR Department = 'D02')(Employee)
RESULT ← π(Pid, PName)((EMP ⋈ AssignedTo ⋈ Project))

```sql
SELECT DISTINCT P.Pid, P.PName
FROM Project P
JOIN AssignedTo A ON P.Pid = A.Pid
JOIN Employee E ON A.Eid = E.Eid
WHERE E.Department IN ('D01', 'D02');
```

	Pid	PName
	101	Banking
	102	University
	103	ShareMarket

3. Find the employees who belong to Delhi and work on either 'University'project or 'ShareMarket' project

P ← σ(PName = 'University' OR PName = 'ShareMarket')(Project)
EMP ← σ(Address = 'Delhi')(Employee)
RESULT ← π(Eid, EName)(EMP ⋈ AssignedTo ⋈ P)

```sql
SELECT DISTINCT E.Eid, E.EName
FROM Employee E
JOIN AssignedTo A ON E.Eid = A.Eid
JOIN Project P ON A.Pid = P.Pid
WHERE E.Address = 'Delhi'
  AND P.PName IN ('University', 'ShareMarket');
```

	Eid	EName
	1	Amit
	3	Jai Prakash
	4	Anita

4. Find the employees who do not work on 'University' project.

UNIV ← π(Eid)(AssignedTo ⋈ σ(PName='University')(Project))
RESULT ← π(Eid, EName)(Employee) – UNIV

```sql
SELECT E.Eid, E.EName
FROM Employee E
WHERE E.Eid NOT IN (
    SELECT A.Eid
    FROM AssignedTo A
    JOIN Project P ON A.Pid = P.Pid
    WHERE P.PName = 'University'
);
```

Eid	EName
2	Rohit
4	Anita
NULL	NULL

5. Find the employees who work on all projects.

RESULT ← π(Eid)(AssignedTo ÷ Project)

```sql
SELECT E.Eid, E.EName
FROM Employee E
WHERE NOT EXISTS (
    SELECT P.Pid
    FROM Project P
    WHERE NOT EXISTS (
        SELECT *
        FROM AssignedTo A
        WHERE A.Eid = E.Eid AND A.Pid = P.Pid
    )
);
```

Eid	EName
NULL	NULL

6. List the employees who have not been assigned any project.

ASSIGNED ← π(Eid)(AssignedTo)
RESULT ← π(Eid, EName)(Employee) – ASSIGNED

```sql
SELECT E.Eid, E.EName
FROM Employee E
WHERE E.Eid NOT IN (SELECT Eid FROM AssignedTo);
```

Eid	EName
NULL	NULL

7. Find the employees who joined the department after the commencement of 'Bank' project.

BANK ← σ(PName='Bank')(Project)
RESULT ← σ(DateOfJoining > BANK.StartDate)(Employee)

```sql
SELECT E.Eid, E.EName
FROM Employee E
WHERE E.DateOfJoining >
   (SELECT StartDate FROM Project WHERE PName = 'Bank');
```

Eid	EName
1	Amit
3	Jai Prakash
NULL	NULL

8. Display the start and termination date of projects which are allotted to 'Jai Prakash'

JP ← σ(EName='Jai Prakash')(Employee)

RESULT ← π(StartDate, TerminationDate)(JP ⋈ AssignedTo ⋈ Project)

```sql
SELECT P.StartDate, P.TerminationDate
FROM Project P
JOIN AssignedTo A ON P.Pid = A.Pid
JOIN Employee E ON A.Eid = E.Eid
WHERE E.EName = 'Jai Prakash';
```

StartDate	TerminationDate
2019-03-10	2022-05-01
2018-11-05	2023-10-10

P4 Consider the following relations:
Primary Keys are indicated by #
Foriegn Keys are underlined.
Student (Roll#, SName, City, Age, Gender)
Course (Cid#, CName, Semester, Credits, Fee)
Enrollment (Roll, Cid, DateOfReg)

```sql
CREATE TABLE Student (
    Roll INT PRIMARY KEY,
    SName VARCHAR(50),
    City VARCHAR(50),
    Age INT,
    Gender VARCHAR(10)
);
INSERT INTO Student VALUES
(1, 'Anita', 'Delhi', 21, 'Female'),
(2, 'Kunal', 'Noida', 22, 'Male'),
(3, 'Rohit', 'Mumbai', 23, 'Male'),
(4, 'Neha', 'Delhi', 20, 'Female'),
(5, 'Aman', 'Chennai', 24, 'Male');
SELECT *
FROM Student;
```

Roll	SName	City	Age	Gender
1	Anita	Delhi	21	Female
2	Kunal	Noida	22	Male
3	Rohit	Mumbai	23	Male
4	Neha	Delhi	20	Female
5	Aman	Chennai	24	Male
NULL	NULL	NULL	NULL	NULL

```
CREATE TABLE Course (
    Cid INT PRIMARY KEY,
    CName VARCHAR(50),
    Semester INT,
    Credits INT,
    Fee DECIMAL(10,2)
);
INSERT INTO Course VALUES
(101, 'MCA', 4, 24, 60000),
(102, 'MBA', 3, 20, 75000),
(103, 'BCA', 1, 18, 45000);
SELECT *
FROM Course;
```

Cid	CName	Semester	Credits	Fee
101	MCA	4	24	60000.00
102	MBA	3	20	75000.00
103	BCA	1	18	45000.00
NULL	NULL	NULL	NULL	NULL

```sql
CREATE TABLE Enrollment (
    Roll INT,
    Cid INT,
    DateOfReg DATE,
    FOREIGN KEY (Roll) REFERENCES Student(Roll),
    FOREIGN KEY (Cid) REFERENCES Course(Cid)
);
INSERT INTO Enrollment VALUES
(1, 101, '2023-01-10'),
(2, 101, '2023-02-12'),
(3, 102, '2023-03-15'),
(4, 103, '2023-04-18'),
(5, 102, '2023-05-20');
SELECT *
FROM Enrollment;
```

	Roll	Cid	DateOfReg
	1	101	2023-01-10
	2	101	2023-02-12
	3	102	2023-03-15
	4	103	2023-04-18
	5	102	2023-05-20

Now write SQL queries for the following:

1. Find the details of students registered in MCA course.

```
SELECT S.*
FROM Student S
JOIN Enrollment E ON S.Roll = E.Roll
JOIN Course C ON E.Cid = C.Cid
WHERE C.CName = 'MCA';
```

	Roll	SName	City	Age	Gender
	1	Anita	Delhi	21	Female
	2	Kunal	Noida	22	Male

2. Display the total number of students enrolled in MBA.

```
SELECT COUNT(*) AS Total_Students
FROM Enrollment E
JOIN Course C ON E.Cid = C.Cid
WHERE C.CName = 'MBA';
```

	Total_Students
	2

3. Display the names of students who are enrolled in the course in which "Anita" is registered.

```sql
SELECT S2.SName
FROM Student S1
JOIN Enrollment E1 ON S1.Roll = E1.Roll
JOIN Enrollment E2 ON E1.Cid = E2.Cid
JOIN Student S2 ON E2.Roll = S2.Roll
WHERE S1.SName = 'Anita';
```

	SName
	Anita
	Kunal

4. Display the names of all the students with their date of enrollment in the courses.

```sql
SELECT S.SName, E.DateOfReg
FROM Student S
JOIN Enrollment E ON S.Roll = E.Roll;
```

	SName	DateOfReg
	Anita	2023-01-10
	Kunal	2023-02-12
	Rohit	2023-03-15
	Neha	2023-04-18
	Aman	2023-05-20

5. Display the name of the course in which “Kunal” is registered.

```sql
SELECT C.CName
FROM Student S
JOIN Enrollment E ON S.Roll = E.Roll
JOIN Course C ON E.Cid = C.Cid
WHERE S.SName = 'Kunal';
```

	CName
	MCA

6. Display the names of boys-students enrolled in MCA.

```sql
SELECT S.SName
FROM Student S
JOIN Enrollment E ON S.Roll = E.Roll
JOIN Course C ON E.Cid = C.Cid
WHERE S.Gender = 'Male'
  AND C.CName = 'MCA';
```

	SName
	Kunal

7. Display the number of students registered in MCA course who live outside Delhi.

```
SELECT COUNT(*) AS Students_Outside_Delhi
FROM Student S
JOIN Enrollment E ON S.Roll = E.Roll
JOIN Course C ON E.Cid = C.Cid
WHERE C.CName = 'MCA'
  AND S.City <> 'Delhi';
```

	Students_Outside_Delhi
	1

P5 Consider any table of your choice and execute following functions on numeric, character and date type fields :

```
CREATE TABLE Employee (
    EmpID INT,
    EmpName VARCHAR(30),
    Salary INT,
    JoinDate DATE,
    City VARCHAR(20)
);
INSERT INTO Employee VALUES
(1, 'Amit Sharma', 25000, '2022-01-10', 'Delhi'),
(2, 'Reena Gupta', 32000, '2021-11-05', 'Mumbai'),
(3, 'Kunal Singh', 28000, '2020-06-15', 'Delhi'),
(4, 'Riya Kapoor', 45000, '2019-03-20', 'Chennai'),
(5, 'Anita Rao', 15000, '2023-07-25', 'Pune');

SELECT *
FROM Employee;
```

EmpID	EmpName	Salary	JoinDate	City
1	Amit Sharma	25000	2022-01-10	Delhi
2	Reena Gupta	32000	2021-11-05	Mumbai
3	Kunal Singh	28000	2020-06-15	Delhi
4	Riya Kapoor	45000	2019-03-20	Chennai
5	Anita Rao	15000	2023-07-25	Pune

• sqrt( )

```
SELECT EmpName, SQRT(Salary) FROM Employee;
```

	EmpName	SQRT(Salary)
	Amit Sharma	158.11388300841898
	Reena Gupta	178.88543819998318
	Kunal Singh	167.33200530681512
	Riya Kapoor	212.13203435596427
	Anita Rao	122.47448713915891

• power( )

```
SELECT EmpName, POWER(Salary, 2) FROM Employee;
```

	EmpName	POWER(Salary, 2)
	Amit Sharma	625000000
	Reena Gupta	1024000000
	Kunal Singh	784000000
	Riya Kapoor	2025000000
	Anita Rao	225000000

• ceil( )

```
SELECT EmpName, CEIL(Salary / 1000.0) FROM Employee;
```

	EmpName	CEIL(Salary / 1000.0)
	Amit Sharma	25
	Reena Gupta	32
	Kunal Singh	28
	Riya Kapoor	45
	Anita Rao	15

• max( ) • min( ) • avg( ) • count( ) • sum ( )

```sql
SELECT
    MAX(Salary),
    MIN(Salary),
    AVG(Salary),
    SUM(Salary),
    COUNT(*)
FROM Employee;
```

MAX(Salary)	MIN(Salary)	AVG(Salary)	SUM(Salary)	COUNT(*)
45000	15000	29000.0000	145000	5

• exp( )

```sql
SELECT EmpName, EXP(1) FROM Employee;
```

EmpName	EXP(1)
Amit Sharma	2.718281828459045
Reena Gupta	2.718281828459045
Kunal Singh	2.718281828459045
Riya Kapoor	2.718281828459045
Anita Rao	2.718281828459045

• mod()

```sql
SELECT EmpName, MOD(Salary, 5000) FROM Employee;
```

	EmpName	MOD(Salary, 5000)
	Amit Sharma	0
	Reena Gupta	2000
	Kunal Singh	3000
	Riya Kapoor	0
	Anita Rao	0

- floor( )

```sql
SELECT EmpName, FLOOR(Salary / 1000.0) FROM Employee;
```

	EmpName	FLOOR(Salary / 1000.0)
	Amit Sharma	25
	Reena Gupta	32
	Kunal Singh	28
	Riya Kapoor	45
	Anita Rao	15

- abs( )

```sql
SELECT ABS(-Salary) FROM Employee;
```

	ABS(-Salary)
	25000
	32000
	28000
	45000
	15000

- greatest ( )

```sql
SELECT GREATEST(Salary, 30000, 40000) FROM Employee;
```

	GREATEST(Salary, 30000, 40000)
	40000
	40000
	40000
	45000
	40000

- round( )

```sql
SELECT ROUND(Salary / 3, 2) FROM Employee;
```

	ROUND(Salary / 3, 2)
	8333.33
	10666.67
	9333.33
	15000.00
	5000.00

• to_date( )

```
SELECT STR_TO_DATE('15-02-2023', '%d-%m-%Y');
```

STR_TO_DATE('15-02-2023', '%d-%m-%Y')
2023-02-15

• months_between( )

```
SELECT TIMESTAMPDIFF(MONTH, JoinDate, SYSDATE()) AS MonthsWorked
FROM Employee;
```

MonthsWorked
46
48
65
79
27

• last_day( )

```
SELECT Last_Day(JoinDate)
FROM Employee;
```

Last_Day(JoinDate)
2022-01-31
2021-11-30
2020-06-30
2019-03-31
2023-07-31

• add_months( )

```sql
SELECT EmpName, ADDDATE(JoinDate, INTERVAL 2 MONTH)
FROM Employee;
```

EmpName	ADDDATE(JoinDate, INTERVAL 2 MONTH)
Amit Sharma	2022-03-10
Reena Gupta	2022-01-05
Kunal Singh	2020-08-15
Riya Kapoor	2019-05-20
Anita Rao	2023-09-25

• next_day( )

```sql
SELECT EmpName, DATE_ADD(JoinDate, INTERVAL (7 - WEEKDAY(JoinDate)) DAY) AS Next_Monday
FROM Employee;
```

EmpName	Next_Monday
Amit Sharma	2022-01-17
Reena Gupta	2021-11-08
Kunal Singh	2020-06-22
Riya Kapoor	2019-03-25
Anita Rao	2023-07-31

• sysdate

```sql
SELECT SYSDATE();
```

• to_char( )

```sql
SELECT DATE_FORMAT(JoinDate, '%d-%M-%Y') AS FormattedDate
FROM Employee;
```

	FormattedDate
	10-January-2022
	05-November-2021
	15-June-2020
	20-March-2019
	25-July-2023

length( )

```sql
SELECT EmpName, LENGTH(EmpName) FROM Employee;
```

	EmpName	LENGTH(EmpName)
	Amit Sharma	11
	Reena Gupta	11
	Kunal Singh	11
	Riya Kapoor	11
	Anita Rao	9

• substr( )

```sql
SELECT SUBSTRING(EmpName, 1, 4) FROM Employee;
```

	SUBSTRING(EmpName, 1, 4)
	Amit
	Reen
	Kuna
	Riya
	Anit

• lower( )

```sql
SELECT LOWER(EmpName) FROM Employee;
```

LOWER(EmpName)
amit sharma
reena gupta
kunal singh
riya kapoor
anita rao

• initcap( )

```sql
SELECT CONCAT(UPPER(LEFT(EmpName,1)), LOWER(SUBSTRING(EmpName,2)))
FROM Employee;
```

CONCAT(UPPER(LEFT(EmpName,1)), LOWER(SUBSTRING(EmpName,2)))
Amit sharma
Reena gupta
Kunal singh
Riya kapoor
Anita rao

• instr( )

```sql
SELECT EmpName, INSTR(EmpName, 'a') FROM Employee;
```

EmpName	INSTR(EmpName, 'a')
Amit Sharma	1
Reena Gupta	5
Kunal Singh	4
Riya Kapoor	4
Anita Rao	1

• concat( )

```sql
SELECT CONCAT(EmpName, ' from ', City) FROM Employee;
```

CONCAT(EmpName, ' from ', City)
Amit Sharma from Delhi
Reena Gupta from Mumbai
Kunal Singh from Delhi
Riya Kapoor from Chennai
Anita Rao from Pune

• lpad( )

```sql
SELECT LPAD(EmpName, 20, '*') FROM Employee;
```

LPAD(EmpName, 20, '...
*********Amit Sharma
*********Reena Gupta
*********Kunal Singh
*********Riya Kapoor
***********Anita Rao

• rpad( )

```sql
SELECT RPAD(EmpName, 20, '*') FROM Employee;
```

RPAD(EmpName, 20, '*')
Amit Sharma*********
Reena Gupta*********
Kunal Singh*********
Riya Kapoor*********
Anita Rao***********

• ltrim( )

```
SELECT LTRIM('      hello');
```

LTRIM('   hello')
hello

• rtrim( )

```
SELECT RTRIM('hello      ');
```

RTRIM('hello   ')
hello

• replace( )

```
SELECT REPLACE(EmpName, 'a', '@') FROM Employee;
```

REPLACE(EmpName, 'a', '@')
Amit Sh@rm@
Reen@ Gupt@
Kun@l Singh
Riy@ K@poor
Anit@ R@o

P6 Write a PL/SQL code that will reverse a given number like 12345 to 54321.

```
DECLARE
num NUMBER;
rev NUMBER;
BEGIN

num:=&num;
rev:=0;

WHILE num>0 LOOP
rev:=(rev*10) + mod(num,10);
num:=floor(num/10);
END LOOP;

DBMS_OUTPUT.PUT_LINE('Reverse of the number is: ' || rev);
END;
/
```


```
Reverse of the number is: 4321


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.007
```

P7 Write a PL/SQL code to accept emp_id from user and display his / her record. Table: Employee (EmpId, EName, DOJ, DOB, Salary, Address)

```sql
CREATE TABLE Employee (
    EmpId      NUMBER PRIMARY KEY,
    EName      VARCHAR2(30),
    DOJ        DATE,
    DOB        DATE,
    Salary     NUMBER(10,2),
    Address    VARCHAR2(100)
);
INSERT INTO Employee VALUES (101, 'Amit Sharma', DATE '2020-05-12', DATE '1995-04-10', 50000, 'Delhi');
INSERT INTO Employee VALUES (102, 'Neha Gupta', DATE '2021-01-20', DATE '1998-10-02', 45000, 'Mumbai');
INSERT INTO Employee VALUES (103, 'Jai Prakash', DATE '2019-08-15', DATE '1990-12-05', 60000, 'Chennai');

COMMIT;

SELECT *
FROM Employee;
```

	EMPID	ENAME	DOJ	DOB	SALARY	ADDRESS
1	101	Amit Sharma	5/12/2020, 12:00:00 AM	4/10/1995, 12:00:00 AM	50000	Delhi
2	102	Neha Gupta	1/20/2021, 12:00:00 AM	10/2/1998, 12:00:00 AM	45000	Mumbai
3	103	Jai Prakash	8/15/2019, 12:00:00 AM	12/5/1990, 12:00:00 AM	60000	Chennai

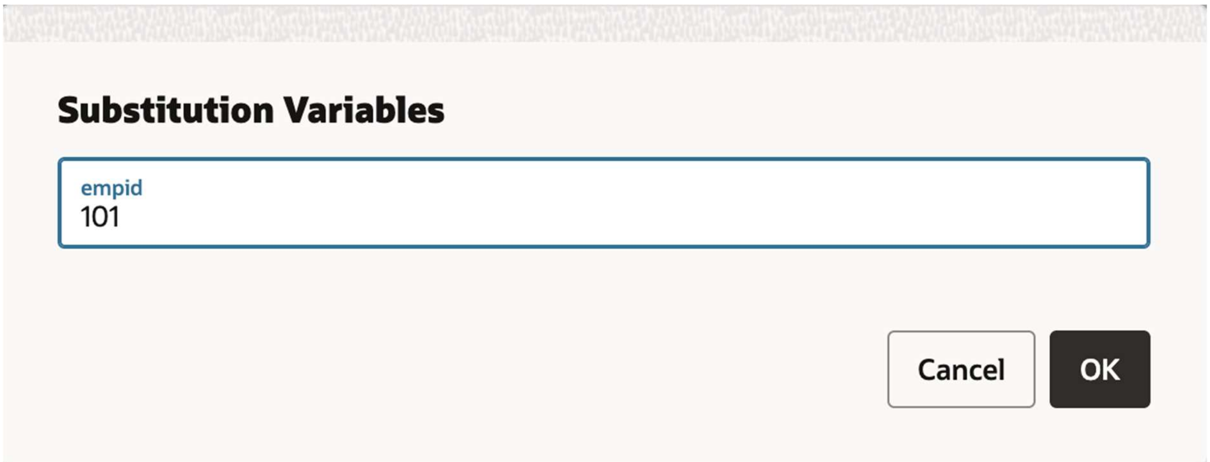
10/2/1998, 12:00:00 AM

```sql
SET SERVEROUTPUT ON;

DECLARE
    v_id Employee.EmpId%TYPE := &empid;
    v_name Employee.EName%TYPE;
    v_doj Employee.DOJ%TYPE;
    v_dob Employee.DOB%TYPE;
    v_salary Employee.Salary%TYPE;
    v_addr Employee.Address%TYPE;
BEGIN
    SELECT EName, DOJ, DOB, Salary, Address
    INTO   v_name, v_doj, v_dob, v_salary, v_addr
    FROM   Employee
    WHERE  EmpId = v_id;

    DBMS_OUTPUT.PUT_LINE('Employee Details:');
    DBMS_OUTPUT.PUT_LINE('ID      : ' || v_id);
    DBMS_OUTPUT.PUT_LINE('Name    : ' || v_name);
    DBMS_OUTPUT.PUT_LINE('DOJ     : ' || v_doj);
    DBMS_OUTPUT.PUT_LINE('DOB     : ' || v_dob);
    DBMS_OUTPUT.PUT_LINE('Salary  : ' || v_salary);
    DBMS_OUTPUT.PUT_LINE('Address : ' || v_addr);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee found with ID = ' || v_id);
END;
/
```




```
Employee Details:
ID      : 101
Name    : Amit Sharma
DOJ     : 12-MAY-20
DOB     : 10-APR-95
Salary  : 50000
Address : Delhi


PL/SQL procedure successfully completed.
```

P8 Write a PL/SQL code that calculates factorial of first five even numbers and stores the same in the given table. Table : Factorial (No, Fact)

```
CREATE TABLE Factorial (
    No   NUMBER,
    Fact NUMBER
);
SET SERVEROUTPUT ON;

DECLARE
    n    NUMBER;
    fac  NUMBER;
BEGIN
    FOR i IN 1..5 LOOP
        n := i * 2;      -- first 5 even numbers: 2,4,6,8,10
        fac := 1;

        -- calculate factorial
        FOR j IN 1..n LOOP
            fac := fac * j;
        END LOOP;

        -- insert into table
        INSERT INTO Factorial VALUES (n, fac);

        DBMS_OUTPUT.PUT_LINE('Inserted: ' || n || ' -> ' || fac);
    END LOOP;

    COMMIT;
END;
/
```

```
Inserted: 2 -> 2
Inserted: 4 -> 24
Inserted: 6 -> 720
Inserted: 8 -> 40320
Inserted: 10 -> 3628800


PL/SQL procedure successfully completed.
```

```sql
SELECT *
FROM Factorial;
```

	NO	FACT
1	2	2
2	4	24
3	6	720
4	8	40320
5	10	3628800

P9 The bank manager of Delhi branch decides to mark the status of all those accounts as 'Inactive' on which there are no transactions performed in last 6 months. Whenever any such update takes place the corresponding record is inserted in another table 'InactiveAccounts' with the name and account no of the account holder and his balance. Table: Account (AccNo, Name, Address, PANNo, Mobile, Status)

```
CREATE TABLE Account (
    AccNo NUMBER PRIMARY KEY,
    Name VARCHAR2(50),
    Address VARCHAR2(50),
    PANNo VARCHAR2(15),
    Mobile VARCHAR2(15),
    Balance NUMBER,
    Status VARCHAR2(10),
    LastTransactionDate DATE
);
```

```
INSERT INTO Account VALUES (101, 'Amit Sharma', 'Delhi', 'ABCDE1234F', '9876543210', 25000, 'Active', DATE '2024-10-10');
INSERT INTO Account VALUES (102, 'Neha Gupta', 'Delhi', 'PQRSX6789L', '9876501020', 50000, 'Active', DATE '2024-05-15');
INSERT INTO Account VALUES (103, 'Ravi Singh', 'Mumbai', 'FHGTY5674Q', '9823456789', 30000, 'Active', DATE '2023-12-20');
INSERT INTO Account VALUES (104, 'Kiran Verma', 'Delhi', 'TYUOP0987L', '9988223344', 45000, 'Active', DATE '2024-02-10');
INSERT INTO Account VALUES (105, 'Pooja Mehta', 'Kolkata', 'LSHDY4532R', '9988112233', 60000, 'Active', DATE '2024-11-01');
```

```
SELECT *
FROM Account;
```

	ACCNO	NAME	ADDRESS	PANNO	MOBILE	BALANCE	STATUS	LASTTRANSACTIONDATE
1	101	Amit Sharma	Delhi	ABCDE1234F	9876543210	25000	Active	10/10/2024, 12:00:00 AM
2	102	Neha Gupta	Delhi	PQRSX6789L	9876501020	50000	Active	5/15/2024, 12:00:00 AM
3	103	Ravi Singh	Mumbai	FHGTY5674Q	9823456789	30000	Active	12/20/2023, 12:00:00 AM
4	104	Kiran Verma	Delhi	TYUOP0987L	9988223344	45000	Active	2/10/2024, 12:00:00 AM
5	105	Pooja Mehta	Kolkata	LSHDY4532R	9988112233	60000	Active	11/1/2024, 12:00:00 AM

```sql
CREATE TABLE InactiveAccounts (
    AccNo NUMBER,
    Name VARCHAR2(50),
    Balance NUMBER,
    InactiveDate DATE
);
```

```
Table INACTIVEACCOUNTS created.
```

```sql
UPDATE Account
SET Status = 'Inactive'
WHERE Address = 'Delhi'
AND LastTransactionDate < ADD_MONTHS(SYSDATE, -6);
```

```
3 rows updated.
```

```sql
CREATE OR REPLACE TRIGGER trg_inactive_accounts
AFTER UPDATE OF Status ON Account
FOR EACH ROW
WHEN (NEW.Status = 'Inactive')
BEGIN
    INSERT INTO InactiveAccounts
    VALUES (:NEW.AccNo, :NEW.Name, :NEW.Balance, SYSDATE);
END;
/
```

```
Trigger TRG_INACTIVE_ACCOUNTS compiled
```

P10 Write a Cursor(PL/SQL code) to display the Employee_name, DateofBirth, and Designation whose basic salary is greater than 15000, if not found then show the proper error message. (Use Exception handing) Table : Employee(EmpId, EName, DOJ, DOB, Salary, Address)

```
CREATE TABLE Employee (
    EmpId NUMBER PRIMARY KEY,
    EName VARCHAR2(50),
    DOJ DATE,
    DOB DATE,
    Salary NUMBER,
    Address VARCHAR2(100),
    Designation VARCHAR2(50)
);
```

```
INSERT INTO Employee VALUES(101, 'Amit Sharma', DATE '2020-03-10', DATE '1995-05-12', 18000, 'Delhi', 'Manager');
INSERT INTO Employee VALUES(102, 'Neha Gupta', DATE '2019-07-18', DATE '1993-11-20', 14000, 'Mumbai', 'Clerk');
INSERT INTO Employee VALUES(103, 'Ravi Singh', DATE '2018-01-25', DATE '1990-02-10', 25000, 'Delhi', 'Developer');
INSERT INTO Employee VALUES(104, 'Kiran Verma', DATE '2021-06-09', DATE '1998-09-15', 15500, 'Chennai', 'Analyst');
INSERT INTO Employee VALUES(105, 'Pooja Mehta', DATE '2017-11-30', DATE '1992-12-05', 12000, 'Kolkata', 'Clerk');

SELECT *
FROM Employee;
```

	EMPID	ENAME	DOJ	DOB	SALARY	ADDRESS	DESIGNATION
1	101	Amit Sharma	3/10/2020, 12:00:00 AM	5/12/1995, 12:00:00 AM	18000	Delhi	Manager
2	102	Neha Gupta	7/18/2019, 12:00:00 AM	11/20/1993, 12:00:00 AM	14000	Mumbai	Clerk
3	103	Ravi Singh	1/25/2018, 12:00:00 AM	2/10/1990, 12:00:00 AM	25000	Delhi	Developer
4	104	Kiran Verma	6/9/2021, 12:00:00 AM	9/15/1998, 12:00:00 AM	15500	Chennai	Analyst
5	105	Pooja Mehta	11/30/2017, 12:00:00 AM	12/5/1992, 12:00:00 AM	12000	Kolkata	Clerk

```sql
DECLARE
    -- Cursor for employees with salary > 15000
    CURSOR emp_cur IS
        SELECT EName, DOB, Address AS Designation
        FROM Employee
        WHERE Salary > 15000;

    v_EName Employee.EName%TYPE;
    v_DOB   Employee.DOB%TYPE;
    v_Desig Employee.Address%TYPE;

    no_data EXCEPTION;
    record_found BOOLEAN := FALSE;

BEGIN
    OPEN emp_cur;
    LOOP
        FETCH emp_cur INTO v_EName, v_DOB, v_Desig;
        EXIT WHEN emp_cur%NOTFOUND;

        record_found := TRUE;

        DBMS_OUTPUT.PUT_LINE(
            'Name: ' || v_EName ||
            ', DOB: ' || TO_CHAR(v_DOB, 'DD-MON-YYYY') ||
            ', Designation: ' || v_Desig
        );
    END LOOP;

    IF NOT record_found THEN
        RAISE no_data;
    END IF;

    CLOSE emp_cur;

EXCEPTION
    WHEN no_data THEN
        DBMS_OUTPUT.PUT_LINE('No employee has salary greater than 15000.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Unexpected Error: ' || SQLERRM);
END;
/
```

```
Name: Amit Sharma, DOB: 12-MAY-1995, Designation: Delhi
Name: Ravi Singh, DOB: 10-FEB-1990, Designation: Delhi
Name: Kiran Verma, DOB: 15-SEP-1998, Designation: Chennai


PL/SQL procedure successfully completed.
```

P11 Create a function that accepts emp id from the calling procedure and returns his salary. Table: Employee (EmpId, EName, DOJ, DOB, Salary, Address)

```
CREATE OR REPLACE FUNCTION get_salary(p_empid NUMBER)
RETURN NUMBER
IS
    v_salary Employee.Salary%TYPE;
BEGIN
    SELECT Salary
    INTO v_salary
    FROM Employee
    WHERE EmpId = p_empid;

    RETURN v_salary;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee ID not found.');
        RETURN NULL;
END;
/
CREATE OR REPLACE PROCEDURE show_salary(p_empid NUMBER)
IS
    v_sal NUMBER;
BEGIN
    v_sal := get_salary(p_empid);

    IF v_sal IS NOT NULL THEN
        DBMS_OUTPUT.PUT_LINE('Salary of Employee ' || p_empid || ' is: ' || v_sal);
    END IF;

END;
/
EXEC show_salary(101);
```

```
Salary of Employee 101 is: 18000

PL/SQL procedure successfully completed.
```

P12 Write a PL/SQL function ODDEVEN to return value TRUE if the number passed to it is EVEN else will return FALSE

```
DECLARE
    n NUMBER := 1634;
    r NUMBER;
BEGIN
    r := MOD(n, 2);

    IF r = 0 THEN
      dbms_output.Put_line('TRUE');
    ELSE
      dbms_output.Put_line('FALSE');
    END IF;
END;
/
```

```
TRUE


PL/SQL procedure successfully completed.
```